

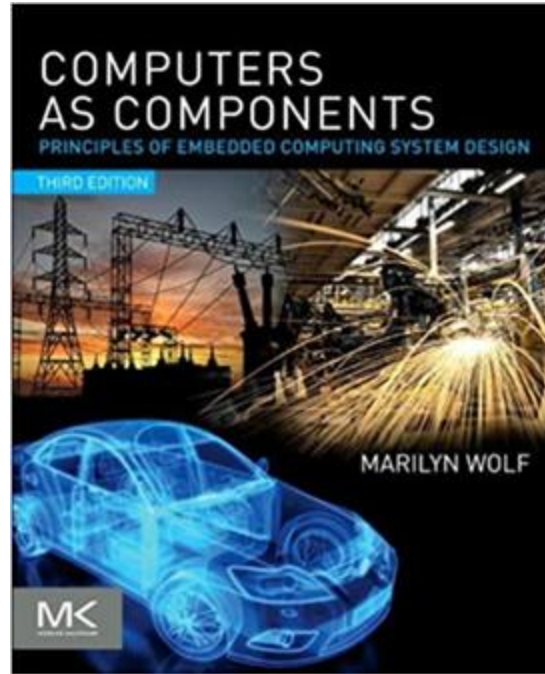
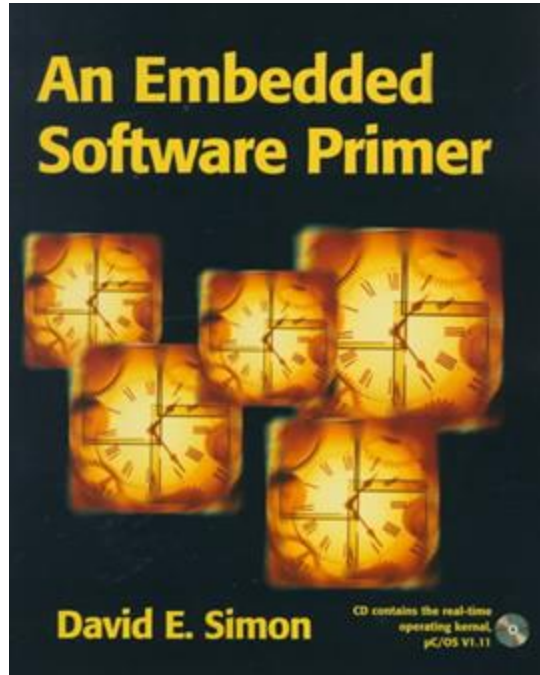


Embedded Software for the Internet Of Things

Embedded Software Testing

Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

Some References for This Lecture



Modern embedded systems

- From really simple to more and more sophisticated systems.



Complex Applications

- Designing these systems **by hand** is no longer feasible.
 - We can't simply wire a few parts together, hack out a bit of software, and spend days or weeks trying to debug the system.



Embedded Systems Life Cycle

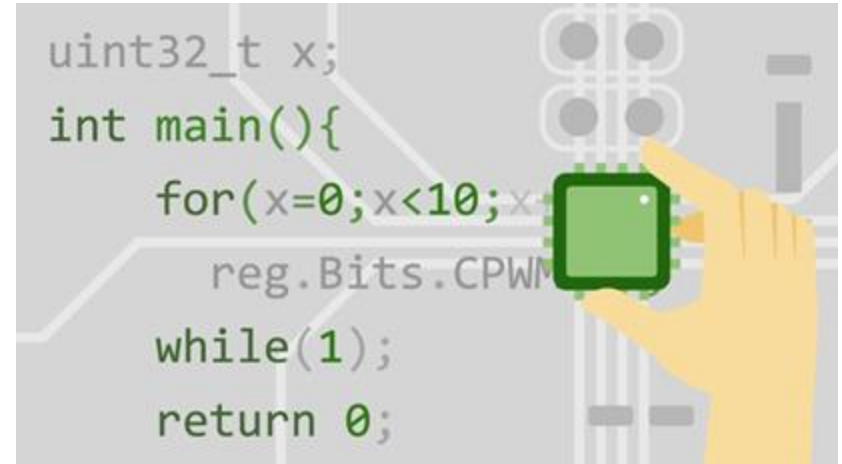
- **Hardware life cycle**

- the design, development, and test of the physical system architecture
 - packaging, printed circuit boards, individual components.



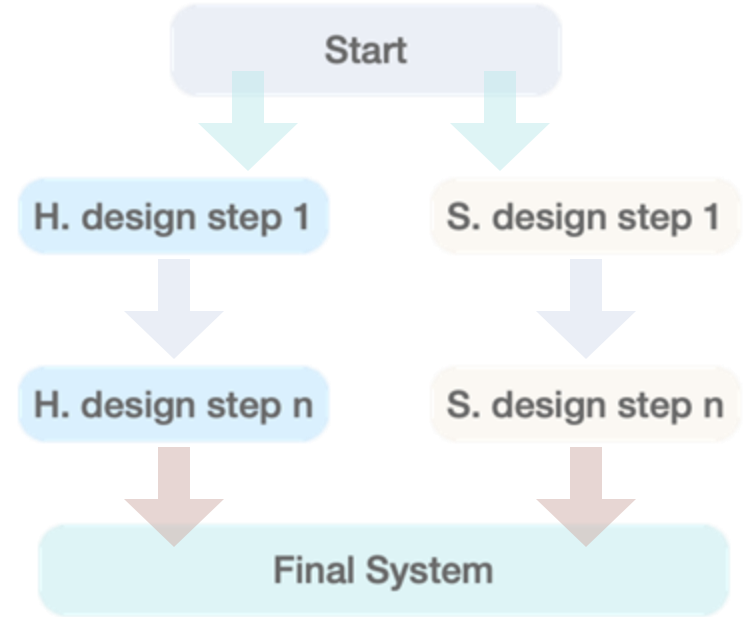
Embedded Systems Life Cycle

- **The software portion:**
 - Algorithmic portion of the application
 - written in a high-level language such as C, or assembler, or a mixture
 - requires detailed knowledge of the microprocessor architecture



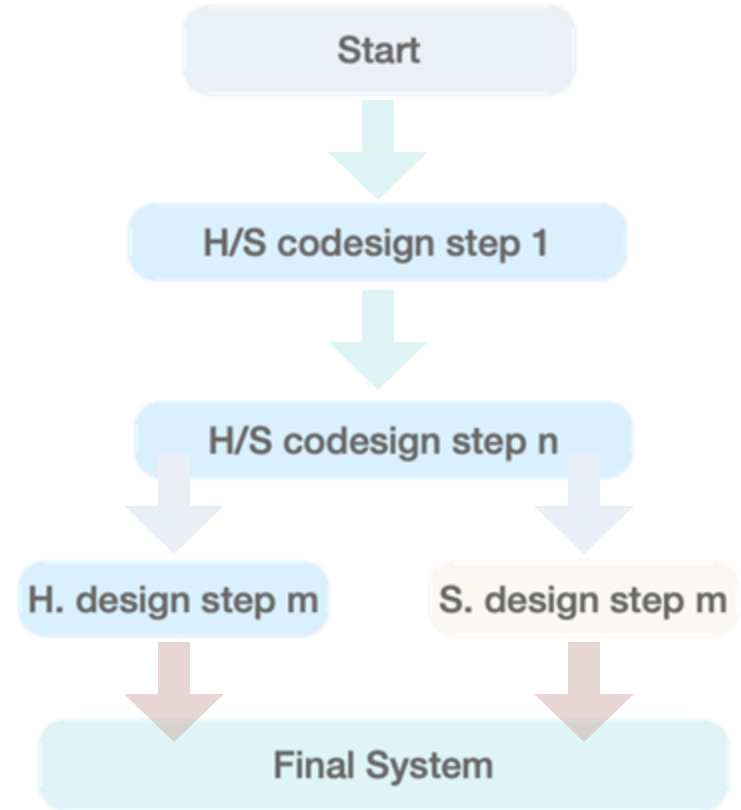
Traditional Design Approach

- The traditional design approach:
 - Design the hardware components.
 - Design the software components.
 - Bring the two together.
 - Spend time testing and debugging the system.



Hardware/Software Co-Design

- **Simultaneous** design of both the hardware and the software components
 - Meeting system-level requirements through **trade-offs** between these two.
 - Which part in HW?
 - Which part in SW?
- An **iterative approach** to the design of hardware and software.

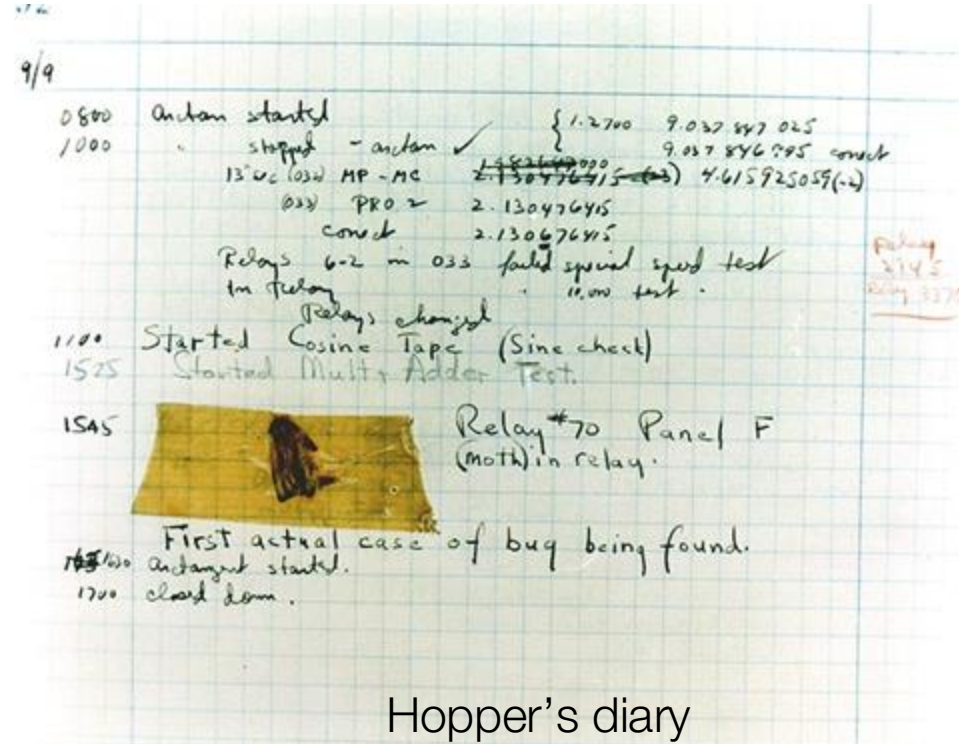


Write Good Code!

- Modular design
- Clean Interfaces
- Commented source files
- But we need more **tools**:
 - to **model** our designs
 - to perform **simulations**
 - to simplify and interactively **optimize** the hardware, software, and firmware.

The First Computer Bug

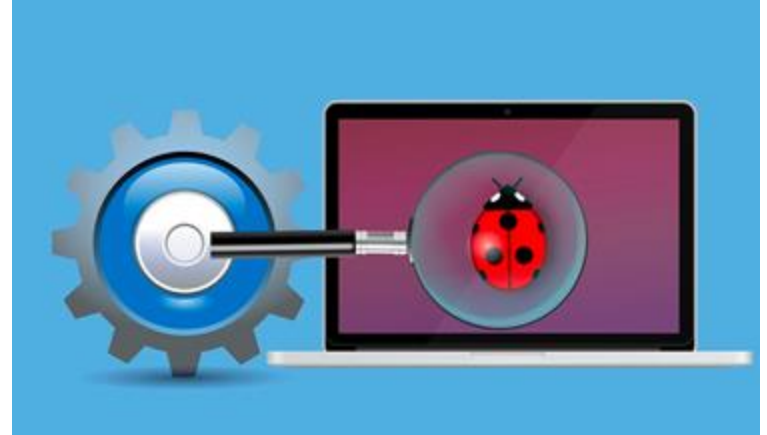
- Recorded by computer scientist Grace Hopper
 - colleagues at Harvard discovered that the **insect** trapped in a relay was causing errors in the computer
- They '**de-bugged**' the machine to fix the problem



The infamous moth was sellotaped

Testing and Debugging

- **Testing**
 - to verify that any hardware or software or the final integrated system perform as it was designed.
- **Debugging**
 - the process of identifying errors in a new or modified hardware or software

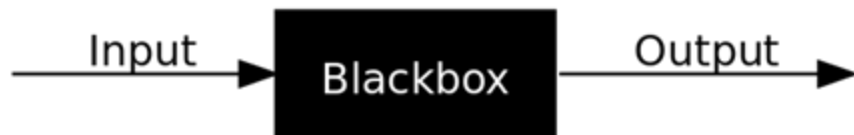


Testing

- Ensure that the system works as it is intended.
 - How much testing is enough?
 - Cannot test the program for every possible combination of inputs.
- The two major types of testing strategies:
 - **Black-box**
 - tests without looking at the internal structure
 - **Clear-box** (also known as **white-box**)
 - generate tests based on the program structure

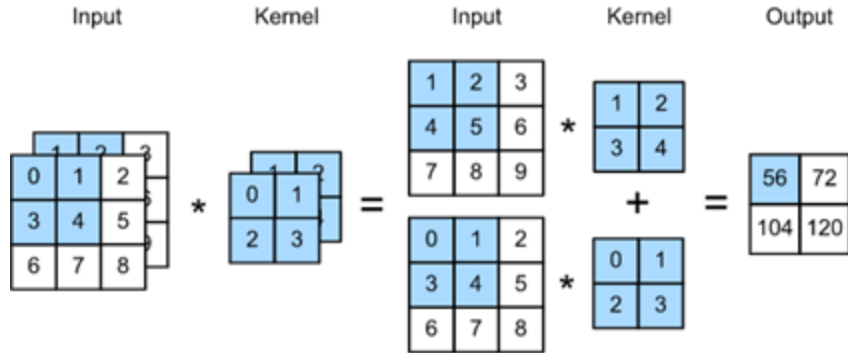
Black-Box Testing

- Testing **without knowledge** of the code being tested.
 - Likely to uncover errors that are unlikely to be found by tests extracted from the code structure.



Black-Box Testing - Example

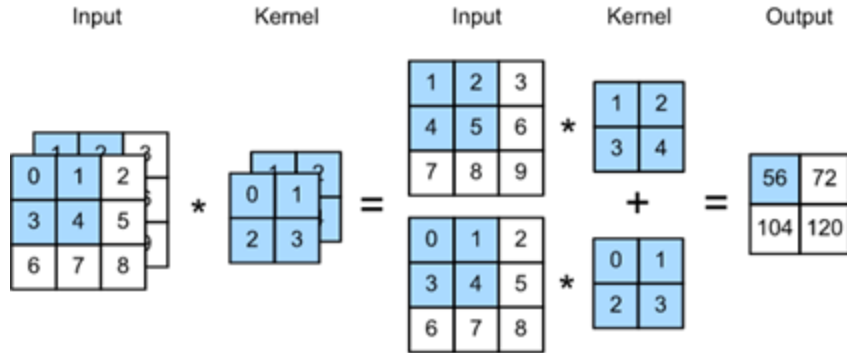
- **Convolution** - A Fundamental Block in Deep Neural Networks



```
uint8_t convolve(uint8_t x, uint8_t y, uint8_t mat[][m_cols][dim], uint8_t kernel[][k_cols][dim]) {  
    ...  
    ...  
}
```

Black-Box Testing - Example

- **Convolution** - A Fundamental Block in Deep Neural Networks



```
uint8_t convolve(uint8_t x, uint8_t y, uint8_t mat[][m_c  
    ...  
    ...  
}
```

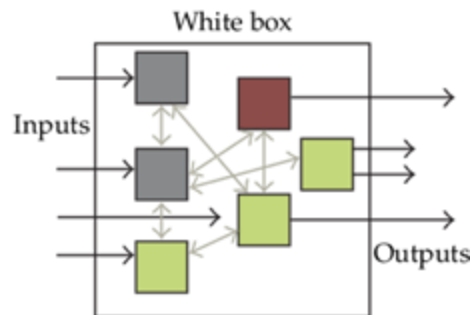
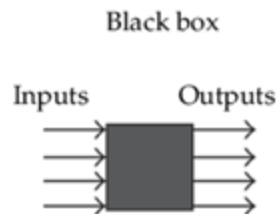
```
uint8_t my_mat [3][3][2] = {{{1,2,3},{4,5,6},{7,8,9}},  
                             {{0,1,2},{3,4,5},{6,7,8}}};  
  
uint8_t my_kernels [2][2][2] = {{{1,2},{3,4}},  
                                 {{0,1},{2,3}}};  
  
void main () {  
    if (convolve(0,0,my_mat,my_kernels) != 56)  
        printf("FAIL!");  
    if (convolve(0,1,my_mat,my_kernels) != 72)  
        printf("FAIL!");  
    if (convolve(1,0,my_mat,my_kernels) != 104)  
        printf("FAIL!");  
    if (convolve(1,1,my_mat,my_kernels) != 120)  
        printf("FAIL!");  
}
```

Black-Box Testing - Test Vectors

- Random inputs
 - May weight distribution based on software specification.
- Regression tests
 - Tests of previous versions, bugs, etc
 - Ensure that previously developed and tested software still performs after a change.

ClearBox Testing

- We must test **both control** and **data operations** of the program
 - control variables in the program and observe the results of computations
- We may need to **modify the program** to **make it more testable**.
- Provide inputs, execute and examine the outputs.
 - Is it possible to execute every path in an arbitrary program?



Controlling and Observing Programs

- The code computes the output of a filter (i.e., **signal processing operation**) from a circular buffer of values:
 - Limits the maximum filter output
- **How to test if this code works correctly?**
 - This code has **controllability** and **observability** problem.

```
firout = 0.0; /* initialize filter output */

/* compute buff*c in bottom part of circular buffer */
for (j = curr, k = 0; j < N; j++, k++)
    firout += buff[j] * c[k];

/* compute buff*c in top part of circular buffer */
for (j = 0; j < curr; j++, k++)
    firout += buff[j] * c[k];

/* limit output value */
if (firout > 100.0) firout = 100.0;
if (firout < -100.0) firout = -100.0;
```

Controlling and Observing Programs

- **Controllability:** How to test if **the limiting code works?**
 - Generate two out-of-range values for firout: positive and negative.
 - Fill the FIR filter circular buffer with N values in the proper range.
- You should write your code so that it will be easy to do these.
 - i.e., easy to control the flow!

```
firout = 0.0; /* initialize filter output */  
  
/* compute buff*c in bottom part of circular  
buffer */  
for (j = curr, k = 0; j < N; j++, k++)  
    firout += buff[j] * c[k];  
  
/* compute buff*c in top part of circular buffer  
*/  
for (j = 0; j < curr; j++, k++)  
    firout += buff[j] * c[k];  
  
/* limit output value */  
if (firout > 100.0) firout = 100.0;  
if (firout < -100.0) firout = -100.0;
```

Just for testing purposes: We need a way
to fill in the buffer or modify firout.



Controlling and Observing Programs

- **Observability:** We should be able to look at the value of `firout` before the limiting code executes.
- We could use a **debugger** to set **breakpoints** in the code:
 - awkward way to perform a large number of tests.
- We would have to add a mechanism for observing **`firout`** independently.

```
firout = 0.0; /* initialize filter output */

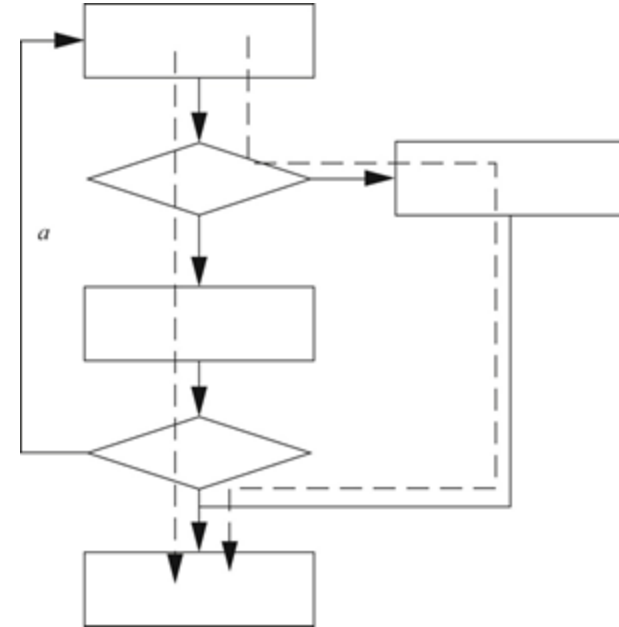
/* compute buff*c in bottom part of circular
buffer */
for (j = curr, k = 0; j <N; j++, k++)
    firout += buff[j] * c[k];

/* compute buff*c in top part of circular buffer
*/
for (j = 0; j <curr; j++, k++)
    firout += buff[j] * c[k];

/* limit output value */
if (firout >100.0) firout = 100.0;
if (firout <-100.0) firout = -100.0;
```

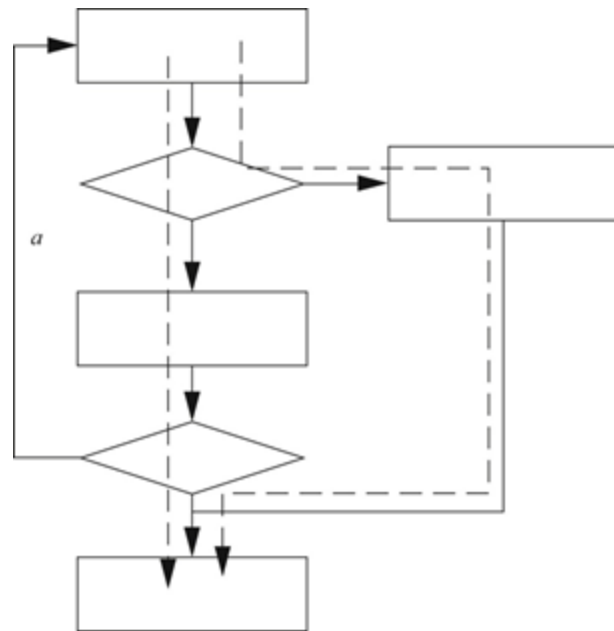
Execution Paths and Testing

- We want to test the program by forcing the program to execute along chosen paths.
 - By giving it inputs that cause it to take the appropriate branches
- Is it possible to execute **every** complete path in an arbitrary program?
 - NO!
- The choice of an appropriate subset of paths to test requires some thought.



Execution Paths and Testing

- Two reasonable ways to choose a set of paths:
 - execute every statement at least once;
 - execute every direction of a branch at least once.
- Ensure that we have executed along every edge in the CDFG.



Debugging Techniques

- The code with lots of bugs in it will be eventually shipped with lots of bugs in it.
 - Write software with fewer bugs in the first place



Debugging Embedded Systems

- **Difficult** and **time-consuming** for embedded systems
 - The fewer bugs, the less trouble!
- The world is extremely **intolerant** of buggy embedded systems.
 - No one accepts medical instruments that stop working during surgery



NASA Pathfinder - MARS 1997

- Collecting **meteorological data**,
- After a few days, the system began resetting itself—a loss of the collected data.
- **Software problem**
 - Watchdog timer 😊
 - Lower priority tasks hold resources needed by the higher priority (important) tasks.



Ariane 5 - European Space Agency

- Designed and built over a **10-year period** at a cost of over **\$7 billion**.
- June 4th, 1996, within **less than a minute** following its initial launch, the rocket was gone.
- The problem was **register overflow**.



Testing on Your Host Machine

- Find the bugs early in the development process saves time and money.
- The target system very often is not available early in the process
 - the hardware may be buggy and unstable



Testing on Your Host Machine

- Embedded systems deals with **unlikely situations**
 - Impossible to test on the target
 - Extremely difficult to make these things happen in the lab
- E.g., code that handles paper jam
 - To run this code, you may require that you jam the paper and then press the button in a millisecond...



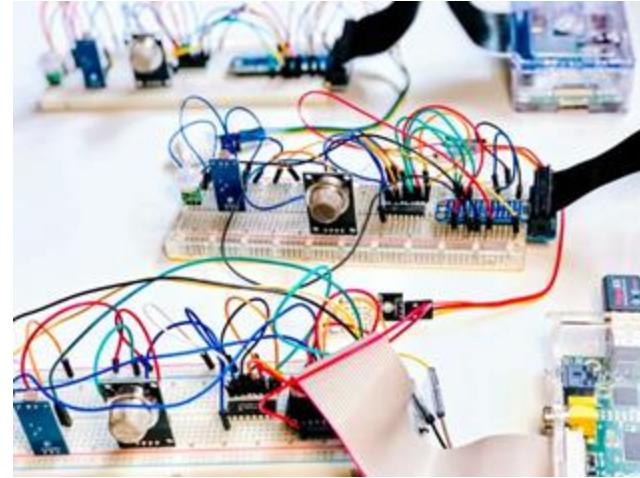
Testing on Your Host Machine

- Develop reusable, repeatable tests.
 - see a bug and let it **happen again**
- Often difficult to create repeatable tests in the target environments
- E.g., cordless barcode scanner has a bug:
 - arises when the user pushes the trigger button while the previous scan is acknowledged -> **difficult to recreate, find and fix it.**



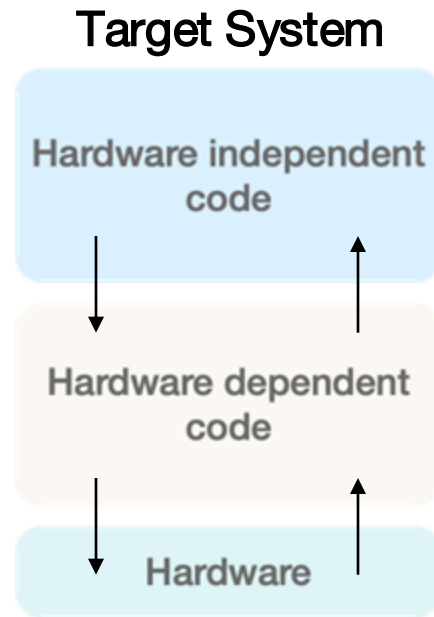
Rule of Thumb

- Don't test on the target more than you have to.
- **Test as much as you can on your development host.**
 - Helps to test your code in a friendlier environment



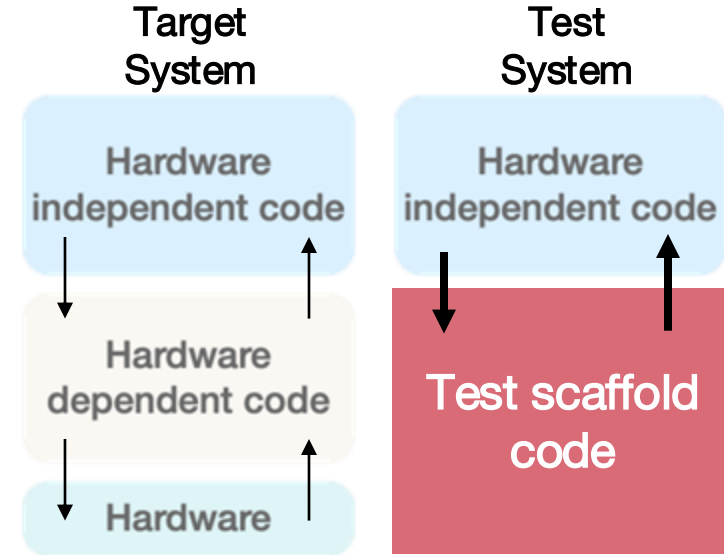
Basic Technique

- Divide your software **into two**:
 - a part that is **independent of the hardware**,
 - another part that is **dependent** on the hardware
 - Put a **clean interface** between the two



Basic Technique

- Replace the hardware-dependent code on with **test scaffold code**
 - The scaffold code **is not as part of the application**
 - supports the process of unit and integration testing
 - provides the same entry points as does the hardware-dependent code
 - calls the same functions in the hardware-independent code



Example - radio.c

- We would like to **test** the procedure **vRadiotask**
 - calls **vTurnonTransmitter** and **vTurnOffRadio** defined in **radiohw.c**.

```
/* File: radio.c */
void vRadioTask (void) {
    ...
    if (!! time to send something on the radio)
    {
        vTurnOnTransmitter(FREQ_NORMAL);
        !! Send data out
        vTurnOffRadio();
    }
}
```

Example - radiohw.c

```
void vTurnonTransmitter (int iFrequencyValue){
    int i;

    disable_interrupts ();
    /* Turn on main power for the radio. */
    ...
    enable_interrupts ();

    /* Shift the frequency value out to the hardware. */
    for (i=0 ; i<16 ;++i){
        /* Send out the lowest bit of iFrequencyValue */
        if (iFrequencyValue & 0x0001){
            /* Put a '1 ' on the data line; pulse the clock line. */
            outp (FRQ_CONROL, DATA_1 & CLOCK_LOW);
            outp (FRQ_CONROL, DATA_1 & CLOCK_HIGH);
        }
        else {
            /* Put a '0" on the data line; pulse the clock line. */
            outp (FRQ_CONROL, DATA_0 & CLOCK_LOW);
            outp (FRQ_CONROL, DATA_0 & CLOCK_HIGH);
        }
        /* Shift iFrequencyValue to get the next lowest bit. */
        iFrequency Value >>=1;
    }

    disable_interrupts ();
    /* Turn on the receiver.*/
    ...
    enable_interrupts ();
}
```

```
void vTurnoffRadio (void) {
    BYTE byPower; /* Byte read from device controlling power. */

    /* Turn off main power for the radio. */
    disable_interrupts ();
    byPower = inp(PWR_CONTROL_ADDR);
    bypower &=~PWR_CONTROL_RADIO_MAIN;
    outp (PWR_CONTROL_ADDR, byPower);
    enable_interrupts();
}
```

Example - radiohw.c

```
void vTurnonTransmitter (int iFrequencyValue){
    int i;

    disable_interrupts ();
    /* Turn on main power for the radio. */
    ...
    enable_interrupts ();

    /* Shift the frequency value out to the hardware. */
    for (i=0 ; i<16 ;++i){
        /* Send out the lowest bit of iFrequencyValue */
        if (iFrequencyValue & 0x0001){
            /* Put a '1 ' on the data line; pulse the clock line. */
            outp (FRQ_CONROL, DATA_1 & CLOCK_LOW);
            outp (FRQ_CONROL, DATA_1 & CLOCK_HIGH);
        }
        else {
            /* Put a '0" on the data line; pulse the clock line. */
            outp (FRQ_CONROL, DATA_0 & CLOCK_LOW);
            outp (FRQ_CONROL, DATA_0 & CLOCK_HIGH);
        }
        /* Shift iFrequencyValue to get the next lowest bit. */
        iFrequency Value >>=1;
    }

    disable_interrupts ();
    /* Turn on the receiver.*/
    ...
    enable_interrupts ();
}
```

```
void vTurnoffRadio (void) {
    BYTE byPower; /* Byte read from device controlling power. */

    /* Turn off main power for the radio. */
    disable_interrupts ();
    byPower = inp(PWR_CONTROL_ADDR);
    bypower &=~PWR_CONTROL_RADIO_MAIN;
    outp (PWR_CONTROL_ADDR, byPower);
    enable_interrupts();
}
```

**Hardware
Dependent**

Example - test.c

- Simulate **outp** and **inp**
 - return **reasonable values** when you are running the test program
- What are the reasonable values?
 - Return the actual values from the hardware which is not likely to be easy

```
/* File: test.c */
void outp (int iAddress, BYTE byData){
    #ifdef LET_USER_SIMULATE_HARDWARE
        printf ("Program wrote %02x to %04x",byData,iAddress) ;
    #endif
    #ifdef SIMULATE_HARDWARE
        !! Remember that software wrote byData to iAddress
        !! Update state of simulated hardware.
    #endif
}

BYTE inp (int iAddress){
    int iData:

    #ifdef LET_USER_SIMULATE_HARDWARE
        printf ("Program needs value for %04x. Enter value", iAddress);
        scanf ("%x", iData) ;
    #endif
    #ifdef SIMULATE_HARDWARE
        !! Figure out what the real hardware would return
    #endif

    return ((BYTE) iData);
}
```

Better Plan for Testing

- Remove **radiohw.c** entirely
 - The test scaffold replaces **vturnontransmitter** and **vTurnoffRadio**.
- Easy to write and interpret if vRadiotask turns on the transmitter when it is appropriate
 - it uses the correct frequency, and it turns the radio off afterward

```
/* File: test.c */

static BOOL fRadioOn;
static int iRadioFrequencyValue;

void vTurnontransmitter (int iFrequencyValue) {
    /* Record the state of the radio. */
    fRadioOn = TRUE;
    iRadioFrequencyValue = iFrequencyValue;
    /* Tell the user */
    printf("Radio turned on with frequency %04x", iFrequencyValue);
}

void vTurnoffRadio (void) {
    /* Record the state of the radio. */
    fRadioOn = FALSE;
    /* Tell the user */
    printf("Radio now off");
}
```

```
/* File: radio.c */
void vRadioTask (void) {
    ...
    if (!! time to send something on the radio) {
        vTurnOnTransmitter(FREQ_NORMAL);
        !! Send data out
        vTurnOffRadio();
    }
}
```

Better Plan for Testing

- This test scaffold **does not test** the code in **radiohw.c**
- Testing **radiohw.c** on the host **makes little sense.**
- You need to test and fix it **on the target.**

```
/* File: test.c */

static BOOL fRadioOn;
static int iRadioFrequencyValue;

void vTurnontransmitter (int iFrequencyValue) {
    /* Record the state of the radio. */
    fRadioOn = TRUE;
    iRadioFrequencyValue = iFrequencyValue;
    /* Tell the user */
    printf("Radio turned on with frequency %04x", iFrequencyValue);
}

void vTurnoffRadio (void) {
    /* Record the state of the radio. */
    fRadioOn = FALSE;
    /* Tell the user */
    printf("Radio now off");
}
```

```
/* File: radio.c */
void vRadioTask (void) {
    ...
    if (!! time to send something on the radio) {
        vTurnOnTransmitter(FREQ_NORMAL);
        !! Send data out
        vTurnOffRadio();
    }
}
```

Test with ISRs

- Interrupt routines tend to divide into **two parts**:
 - one that deals with the hardware
 - one that deals with the rest of the system.
- **Structure your interrupt routines** so that the hardware-dependent part calls the **hardware independent part**.

```
/* File: serial.c */
#define CR OX0d
#define SIZEOF_CMD_BUFFER 200
BYTE a_byCommandBuffer[SIZEOF_CMD_BUFFER];

/* Queue to send message to command-handling task. */
extern unsigned long qidcommands;

void interrupt vHandleRxHardware (void){
    BYTE byChar;
    /* The character we received. */
    int iHwError;
    /* Hardware error, if any */
    iHwError = !! Get status from hardware;
    if (iHwError == CHARACTER_RXD_OK){
        /* We received a character; deal with it. */
        byChar = !! Read byte from hardware;
        vHandleRxByte (byChar);
    }
    else
        !! Deal with hardware error

    !! Reset the hardware as necessary.
    !! Reset interrupt controller as necessary.
}
```

Test with ISRs

- This ISR reads a character from the hardware and calls **vhandleRxByte**
 - Reading byte part is hardware dependent

```
/* File: serial.c */
#define CR OX0d
#define SIZEOF_CMD_BUFFER 200
BYTE a_byCommandBuffer[SIZEOF_CMD_BUFFER];

/* Queue to send message to command-handling task. */
extern unsigned long qidcommands;

void interrupt vHandleRxHardware (void) {
    BYTE byChar;
    /* The character we received. */
    int iHwError;
    /* Hardware error, if any */
    iHwError = !! Get status from hardware;
    if (iHwError == CHARACTER_RXD_OK) {
        /* We received a character; deal with it. */
        byChar = !! Read byte from hardware;
        vHandleRxByte(byChar);
    }
    else
        !! Deal with hardware error

    !! Reset the hardware as necessary.
    !! Reset interrupt controller as necessary.
}
```

Test with ISRs

- **vhandleRxByte**

- no hardware dependency
- puts the character into a circular buffer
- deals with buffer overflows

```
void vHandleRxByte (BYTE byReceived){
    static BYTE *p_byCommandBufferTail = a_byCommandBuffer;
    extern BYTE *p_byCommandBufferHead;
    unsigned long a_ulmessage[4]; /* Message buffer. */

    /* Advance the tail pointer and wrap if necessary */
    ++p_byCommandBufferTail;
    if (p_byCommandBufferTail == &a_byCommandBuffer[SIZEOF_CMD_BUFFER])
        p_byCommandBufferTail = a_byCommandBuffer;

    /* If the buffer is not full. . . . */
    if (p_byCommandBufferTail != p_byCommandBufferHead) {
        /* Store the character in the buffer. */
        p_byCommandBufferTail = byReceived;
        /* we got a carriage return */
        if (*p_byCommandBufferTail == CR) {
            /* Build the message. . . */
            a_ulmessage[0] = MSG_COMMAND_ARRIVED;
            a_ulmessage[1] = 0L;
            a_ulmessage[2] = 0L;
            a_ulmessage[3] = 0L;
            ...
        }
        else{
            /* Discard the character; move the pointer back.*/
            if (p_byCommandBufferTail == a_byCommandBuffer)
                p_byCommandBufferTail = &a_byCommandBuffer[SIZEOF_CMD_BUFFER];
            --p_byCommandBufferTail;
        }
    }
}
```

Test with ISRs

- **vTestMain** routine can test **vHandleRxByte** and its interaction with the system.
- It can create all of those other bug-prone cases that need testing.
- It will not test **vHandleRxHardware**
 - which is hardware-dependent and would be difficult to test

```
/* File: test.c */
void vTestMain (void){
    BYTE a_byTestCommand[]="HELLO WORLD\x0d";
    BYTE *p_by;
    ...
    /* Send each of the characters in a_byTestCommand */
    p_by =a_byTestCommand
    while (*p_by){
        /* Send a single character as though received by the
        interrupt */
        vHandleRxByte(*p_by);
        /* Go to the next character */
        ++p_by;
    }
}
```

Test with ISRs

- **Concurrency bugs!**

- Data shared between interrupts and the foreground program

- What happens if the application is using **a_byCommandBuffer** and interrupt puts a character in the buffer?

- Be careful!

```
/* File: serial.c */
#define CR OX0d
#define SIZEOF_CMD_BUFFER 200
BYTE a_byCommandBuffer[SIZEOF_CMD_BUFFER];

/* Queue to send message to command-handling task. */
extern unsigned long qidcommands;

void interrupt vHandleRxHardware (void) {
    BYTE bychar;
    /* The character we received. */
    int iHwError;
    /* Hardware error, if any */
    iHwError = !! Get status from hardware;
    if (iHwError ==- CHARACTER_RXD_OK) {
        /* We received a character; deal with it. */
        byChar = !! Read byte from hardware;
        vHandleRxByte (byChar);
    }
    else
        !! Deal with hardware error

    !! Reset the hardware as necessary.
    !! Reset interrupt controller as necessary.
}
```

Test Scripts

```
int mx_strcmp(const char *s1, const char *s2) {
    while (*s1) {
        if (*s1 != *s2) {
            break;
        }
        s1++;
        s2++;
    }
    return *s1 - *s2;
}
```

```
#include "tests.c"

int main(void) {
    tests_run_all();
}
```

```
#!/bin/bash
```

```
rm -f app
clear
```

```
clang -std=c11 -Wall -Wextra -Werror -Wpedantic ./main.c -o app
```

```
./app
```

Test Script

```
void test_equal_strings() {
    const char *str1 = "abc";
    const char *str2 = "abc";
    int result = mx_strcmp(str1, str2);

    if (result != 0) {
        test_print("FAILED: test_equal_strings()\n");
    }
}

void test_99() {
    const char *str1 = "abc";
    const char *str2 = "ab";
    int result = mx_strcmp(str1, str2);

    if (result != 99) {
        test_print("FAILED: test_99()\n");
    }
}

void test_minus_100() {
    const char *str1 = "abc";
    const char *str2 = "abcd";
    int result = mx_strcmp(str1, str2);

    if (result != -100) {
        test_print("FAILED: test_minus_100()\n");
    }
}

// Run all

void tests_run_all(void) {
    test_equal_strings();
    test_99();
    test_minus_100();
}
```

Assert MACRO

- To have the program **check things** that you believe **must be true** at any given point in the code
- Takes a single parameter:
 - if the parameter evaluates to TRUE, assert does nothing; otherwise, assert causes the program to stop by printing some useful message

```
#ifdef NDEBUG
    #define assert (bool_expression) /* Define it as nothing */
#else
    #define assert(bool_expression) \
        if (bool_expression) \
            ... \
        else \
            bad_assertion(__FILE__, __LINE__, #bool_expression)
#endif
```

Assert MACRO

- Compiles to no code when you compile the code with **NDEBUG**

```
#include "assert.h"

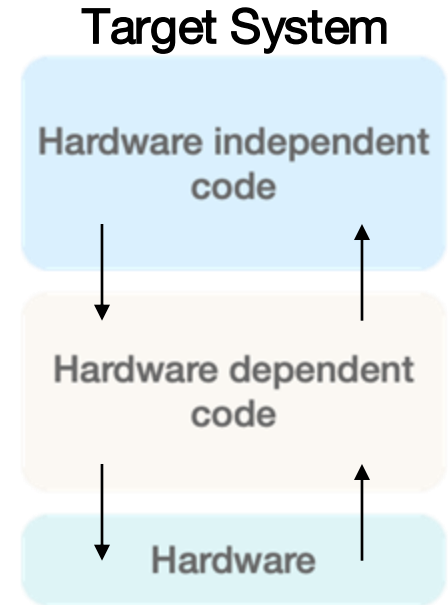
void sendFrame(FRAME *pFrame, BOOL fFrameFromParent, BYTE byMacAddrFrom) {
    assert (pFrame != NULL);
    assert (pFrame->MacHdr.byMplex & MAC_ESSID_HEADER);
    assert (byMacAddrFrom <= ADDR_MAX);
    assert (byMacAddrFrom >= ADDR_MIN);
    assert (byGetcontext () = CONTEXT_TASK);
    switch (MdFindDestination (pFrame)) {
        case DESTINATION_IS_PARENT
            assert (pFrame->byMode & MAC_MODE_USE_STATION);
            assert (pFrame->byMode & MAC_MODE_ASSOCIATED);
        default:
            assert (!"Bad return from MdFindDestination");
    }
}
```

Assert MACRO

- On the target, most embedded systems do not have a display:
 - disable interrupts and **spin in an infinite loop**
 - execute **an illegal instruction to stop the system**
 - turn on and **blink** light-emitting diodes
 - write the values of parameters or dump the contents of stack (or the memory) to some specific memory location

Objections

- Embedded-system code is **very hardware-dependent**.
- It is also true that every embedded system has at least **some hardware-independent code**
- **Most of the code** in most embedded systems has no direct contact with the hardware:
 - it interacts only with the microprocessor.



Objections

- Building a test scaffold is a **trouble**:
 - Finding bugs in the lab takes so much time than developing a test scaffold
- The effort necessary to write this code is **negligible** compared to the effort involved in writing most embedded-system software.

Objections

- You cannot test some important characteristics on the host.
 - interaction with the hardware, response and throughput, shared-data, portability problems...
- Would you prefer working on these issues with a software loaded with other problems?



Let's test our activity recognition code!

Activity Recognition Code

```
void train(features_t *classModel){
    accelWindow sampleWindow;
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_SIZE; ++i) {
        acquire_window(sampleWindow);
        transform(sampleWindow);
        featurize(&features, sampleWindow);

        classModel[i] = features;
    }
}
```

```
void acquire_window(accelWindow window){
    accelReading sample;
    unsigned samplesInWindow = 0;

    while (samplesInWindow < ACCEL_WINDOW_SIZE) {
        accel_sample(&sample);
        window[samplesInWindow++] = sample;
    }
}
```

```
void accel_sample(accelReading* result){
    result->x = readAccelX();
    result->y = readAccelY();
    result->z = readAccelZ();
}
```

Activity Recognition Code

```
void train(features_t *classModel) {
    accelWindow sampleWindow;
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_SIZE; ++i) {
        acquire_window(sampleWindow);
        transform(sampleWindow);
        featurize(&features, sampleWindow);

        classModel[i] = features;
    }
}
```

```
#define SAMPLE_NOISE_FLOOR 10

void transform(accelWindow window)
{
    unsigned i = 0;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        accelReading *sample = &window[i];

        if (sample->x < SAMPLE_NOISE_FLOOR ||
            sample->y < SAMPLE_NOISE_FLOOR ||
            sample->z < SAMPLE_NOISE_FLOOR) {

            sample->x = (sample->x > SAMPLE_NOISE_FLOOR) ? sample->x : 0;
            sample->y = (sample->y > SAMPLE_NOISE_FLOOR) ? sample->y : 0;
            sample->z = (sample->z > SAMPLE_NOISE_FLOOR) ? sample->z : 0;
        }
    }
}
```

Activity Recognition Code

```
void train(features_t *classModel){
    accelWindow sampleWindow;
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_SIZE; ++i)
        acquire_window(sampleWindow);
        transform(sampleWindow);
        featurize(&features, sampleWindow);

    classModel[i] = features;
}
```

```
void featurize(features_t *features, accelWindow aWin){
    accelReading mean;
    accelReading stddev;

    mean.x = mean.y = mean.z = 0;
    stddev.x = stddev.y = stddev.z = 0;
    int i;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        mean.x += aWin[i].x; // x
        mean.y += aWin[i].y; // y
        mean.z += aWin[i].z; // z
    }

    mean.x >>= 2; mean.y >>= 2; mean.z >>= 2;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        stddev.x += aWin[i].x > mean.x ? aWin[i].x - mean.x : mean.x - aWin[i].x; // x
        stddev.y += aWin[i].y > mean.y ? aWin[i].y - mean.y : mean.y - aWin[i].y; // y
        stddev.z += aWin[i].z > mean.z ? aWin[i].z - mean.z : mean.z - aWin[i].z; // z
    }

    stddev.x >>= 2; stddev.y >>= 2; stddev.z >>= 2;

    unsigned meanmag = mean.x*mean.x + mean.y*mean.y + mean.z*mean.z;
    unsigned stddevmag = stddev.x*stddev.x + stddev.y*stddev.y + stddev.z*stddev.z;

    features->meanmag = sqrt16(meanmag);
    features->stddevmag = sqrt16(stddevmag);
}
```

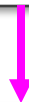
Activity Recognition Code

- Any hardware dependency?
- How to test?

Activity Recognition Code

- Any hardware dependency?
 - **accel_sample**
- How to test?
 - Create a scaffold for **accel_sample**
- **Test on PC!**

```
void accel_sample(accelReading* result) {  
    result->x = readAccelX();  
    result->y = readAccelY();  
    result->z = readAccelZ();  
}
```



```
void accel_sample(accelReading* result) {  
    result->x = produceX();  
    result->y = produceY();  
    result->z = produceZ();  
}
```

Software and Hardware Tools

Voltmeter

- Doubts about the reliability of the hardware on which you are testing your software?
 - a voltmeter for measuring the voltage difference between two points



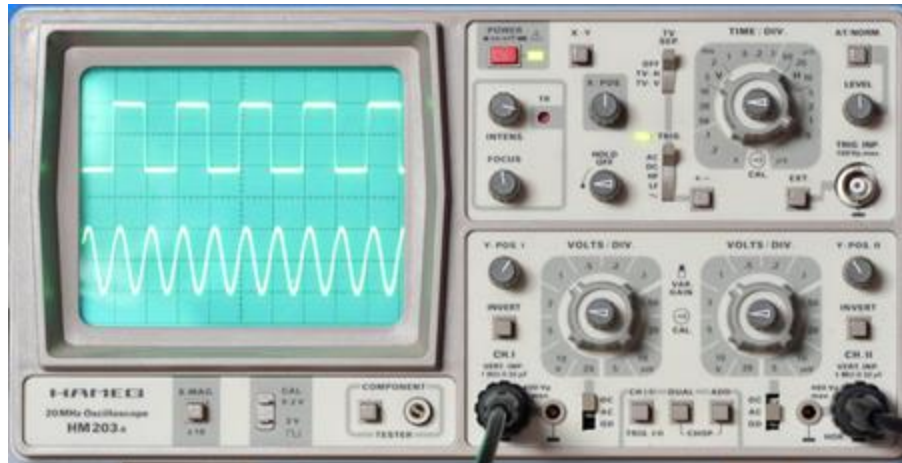
Voltmeter

- The use of voltmeter by software engineers:
 - to determine whether or not the chips in your circuit have power.
 - broken leads, incorrect wiring, etc..
- Put one of the probes on VCC pin, the other on GND pin:
 - if the voltmeter doesn't indicate the right voltage, then you have a hardware problem to fix.



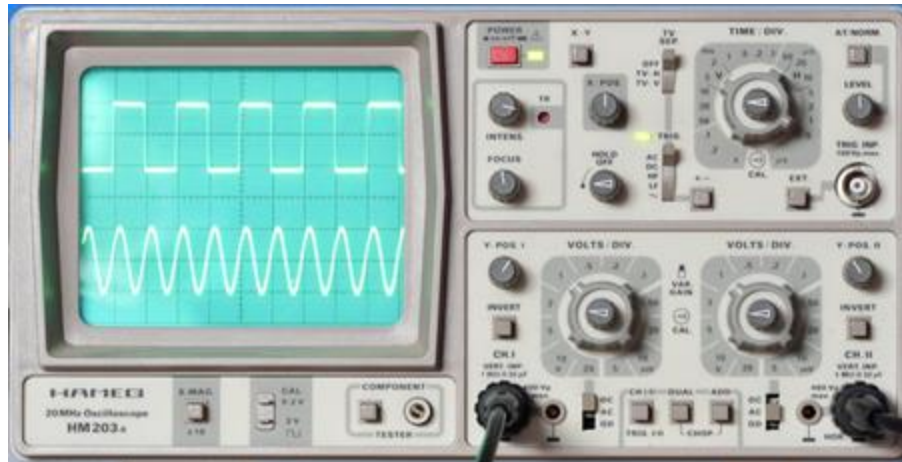
Oscilloscope

- An oscilloscope or scope graphs voltage versus time
 - analog device that detects the signal's exact voltage.
 - can monitor one or two signals simultaneously
 - can adjust the time and voltage scales over a fairly wide range.



Oscilloscope

- The use of oscilloscope by software engineers:
 - As a voltmeter to see if your circuit is working at all:
 - e.g. by watching the MCUs clock input or by watching an address or a data signal SPI, I2C,...



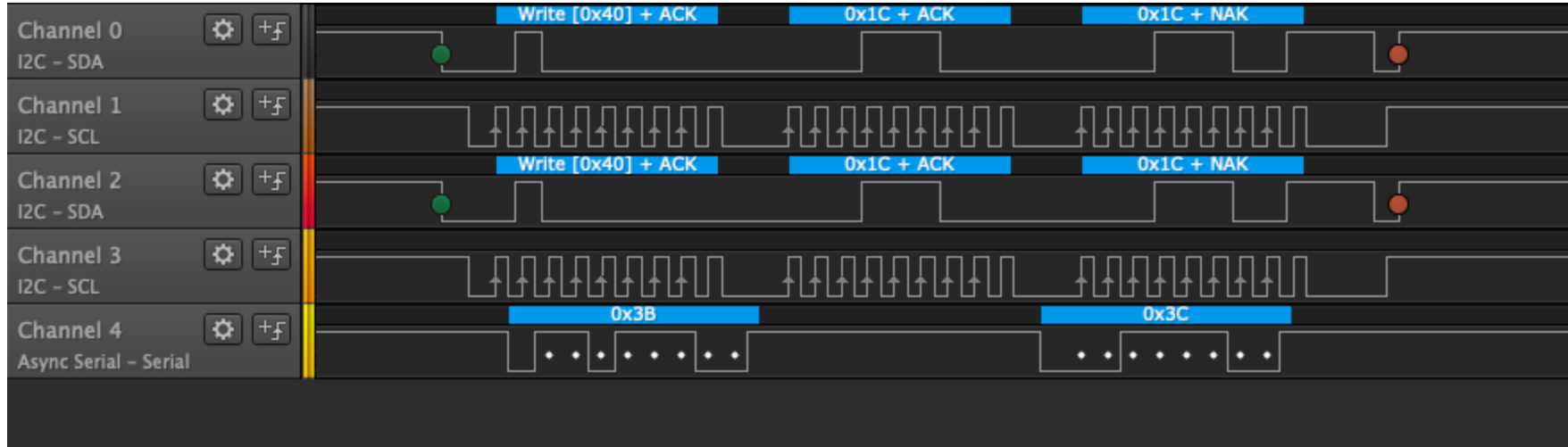
Logic Analyzer

- A logic analyzer captures signals and graphs them on its screen
 - tracks many signals simultaneously
 - only knows about two voltage levels: VCC and ground.
 - If a signal has a voltage somewhere in between VCC and ground, the logic analyzer will report it either as VCC or as ground.



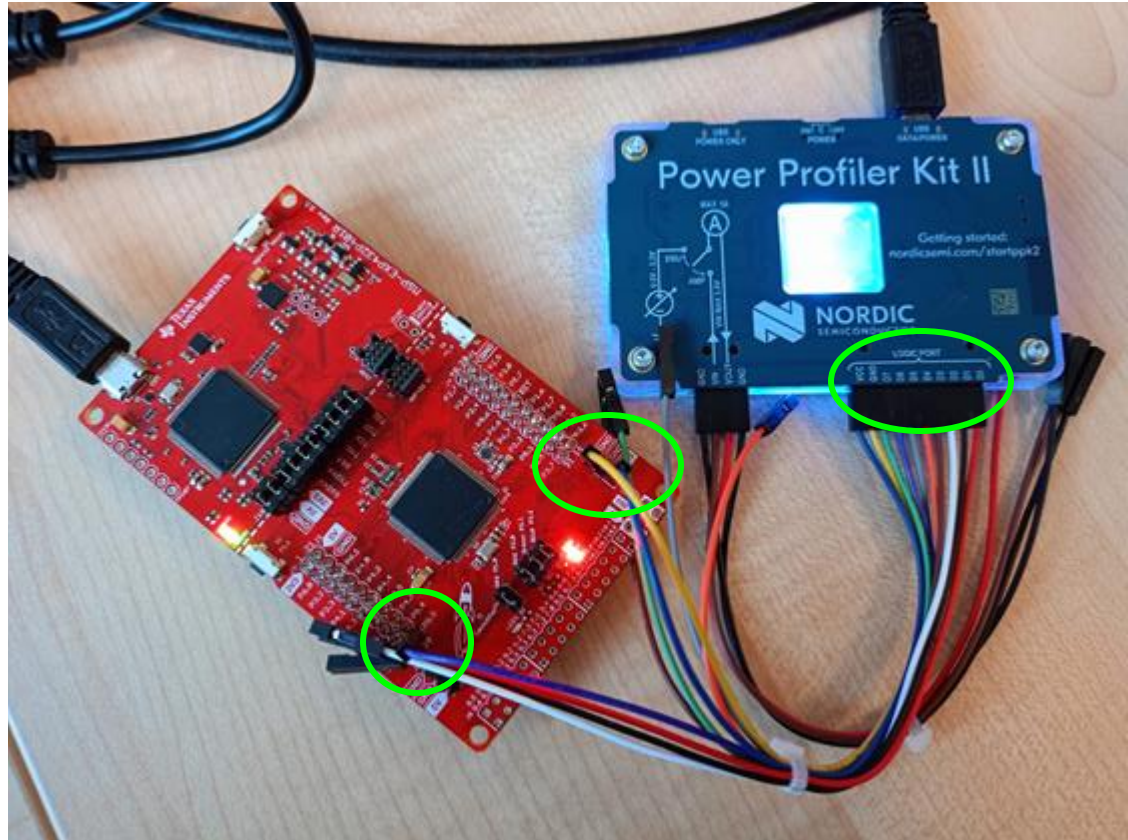
Logic Analyzer

- Signal changes from low to high
 - You can measure how long it takes to respond. e.g. attach your logic analyzer to the pin that generates button interrupt



EXAMPLE - Power Profiler Kit II

- <https://www.nordicsemi.com/Products/Development-hardware/Power-Profiler-Kit-2>



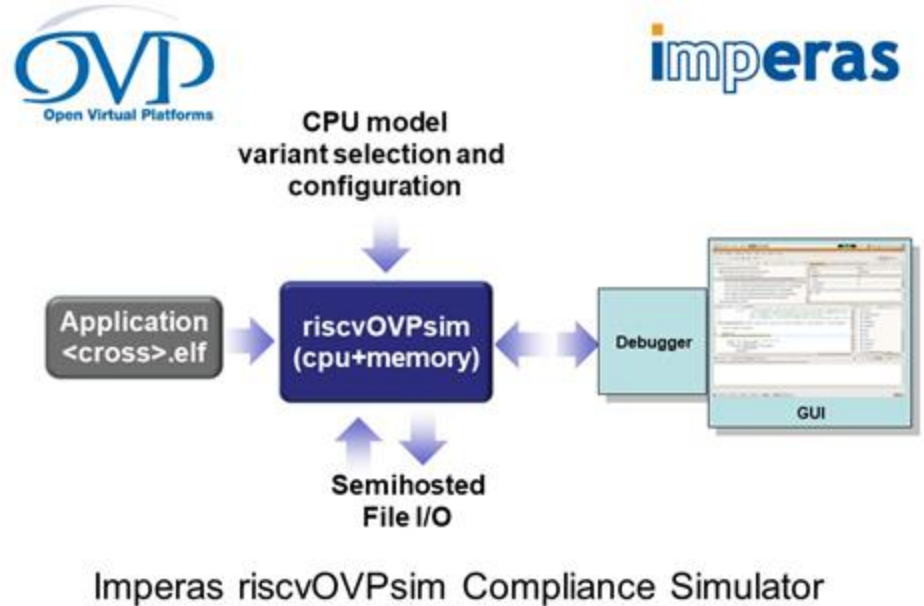
In-Circuit Emulators (ICE)

- Replaces the microprocessor in the target circuit
 - connects to all of the signals to which the real microprocessor attaches and drives them all the same way.
- Gives **debugging capabilities** similar to standard desktop debuggers
 - examine the contents of memory and registers, see the source code, resume execution, or single-step through the code



Instruction Set Simulators

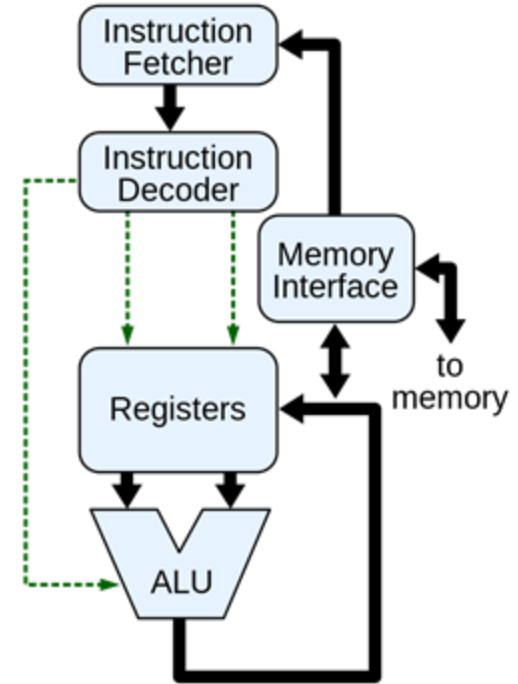
- Simulates the microprocessor architecture and instruction set on your host
 - Reads the instructions
 - Keeps track of the values in the (simulated) RAM
 - Mimic the behavior of the real target hardware.



<https://github.com/riscv/riscv-ovpsim>

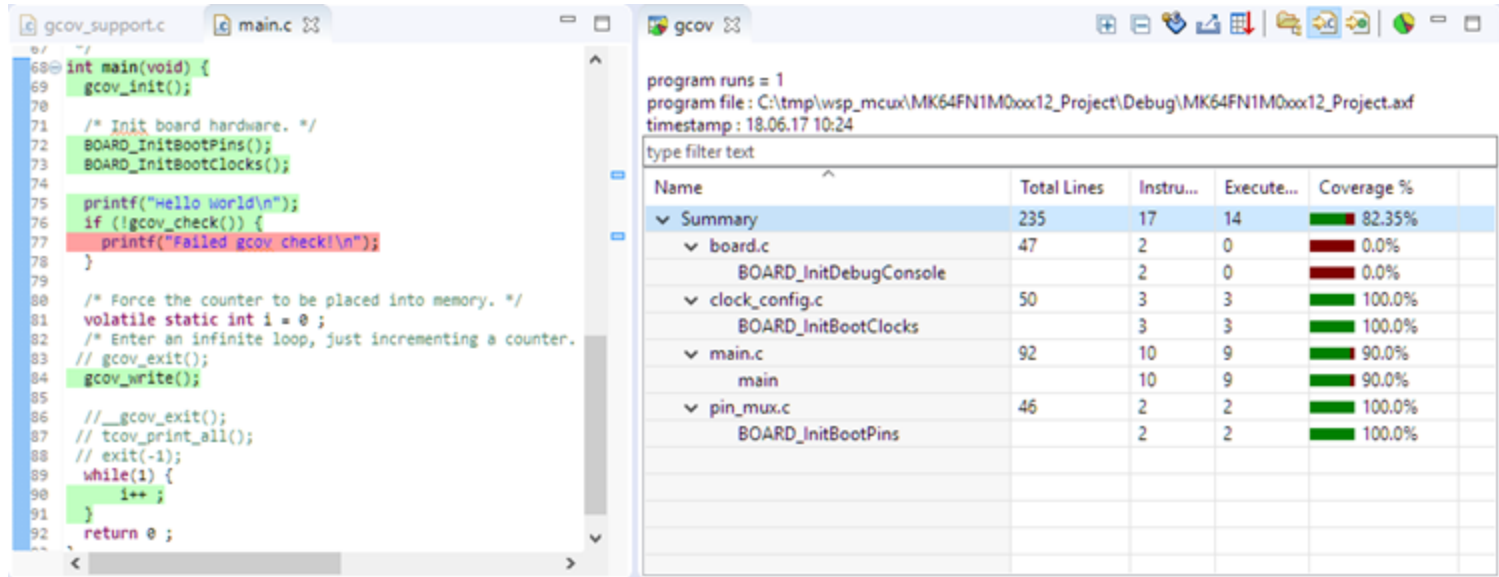
Instruction Set Simulators

- Gives you the **number of instructions** (or target cycles)
 - To determine response and throughput
- Uses the target instruction set
 - **Test assembly-language code**
- Simulate microprocessor's built-in peripherals
 - e.g. built-in timer, when that timer expires, it will call interrupt routine



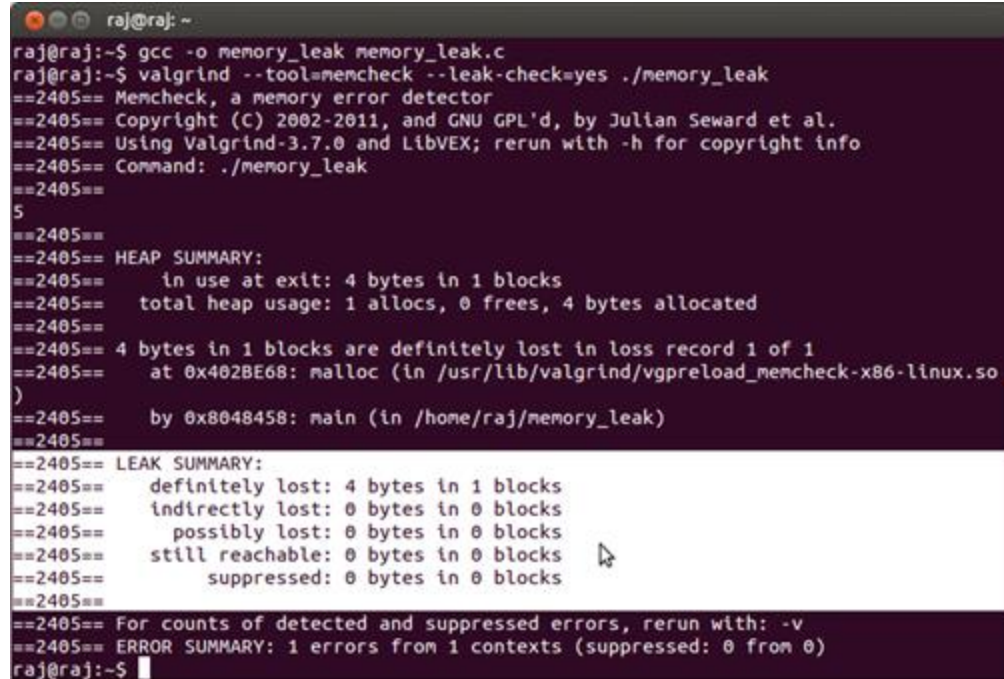
Code Coverage Tools

- **GNU Gcov:** source code coverage analysis and statement-by-statement profiling tool.
 - generates exact counts of the number of times each statement in a program is executed



Dynamic Analysis Tools

- Valgrind (<https://valgrind.org/>)
 - automatically detects many memory management and threading bugs, and profile your programs in detail.



```
raj@raj: ~  
raj@raj:~$ gcc -o memory_leak memory_leak.c  
raj@raj:~$ valgrind --tool=memcheck --leak-check=yes ./memory_leak  
==2405== Memcheck, a memory error detector  
==2405== Copyright (C) 2002-2011, and GNU GPL'd, by Julian Seward et al.  
==2405== Using Valgrind-3.7.0 and LibVEX; rerun with -h for copyright info  
==2405== Command: ./memory_leak  
==2405==  
5  
==2405==  
==2405== HEAP SUMMARY:  
==2405==    in use at exit: 4 bytes in 1 blocks  
==2405== total heap usage: 1 allocs, 0 frees, 4 bytes allocated  
==2405==  
==2405== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1  
==2405==    at 0x402BE68: malloc (in /usr/lib/valgrind/vgpreload_memcheck-x86-linux.so  
)  
==2405==    by 0x8048458: main (in /home/raj/memory_leak)  
==2405==  
==2405== LEAK SUMMARY:  
==2405==    definitely lost: 4 bytes in 1 blocks  
==2405==    indirectly lost: 0 bytes in 0 blocks  
==2405==    possibly lost: 0 bytes in 0 blocks  
==2405==    still reachable: 0 bytes in 0 blocks  
==2405==    suppressed: 0 bytes in 0 blocks  
==2405==  
==2405== For counts of detected and suppressed errors, rerun with: -v  
==2405== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)  
raj@raj:~$
```